

DCIT 202- MOBILE APPLICATION DEVELOPMENT

Session 7 – State Management in React Native

Lecturer: Mr. Paul Nii Tackie Ammah, DCS

Contact Information: pammah@ug.edu.gh



UNIVERSITY OF GHANA

Department of Computer Science
School of Physical and Mathematical Sciences

Session Overview

- Goals and Objectives
 - Understand the fundamental concepts of state and props.
 - Learn how to pass data between components using props effectively.
 - Gain insight into advanced state management strategies including Context API.

NB: Contents of slides largely adapted from <https://reactnative.dev/>

Session Outline

- Managing local state and the role of props in component
- Using Local State Management & Context API



What is State Management

- State management refers to the handling of data within an application that dictates the UI rendering.
- It involves storing, retrieving, and manipulating data to maintain the application's state and respond to user interactions appropriately.
- State can include:
 - variables, data fetched from APIs, user input, and other dynamic information.

Techniques for Simplifying State Management

- **Local State Management:**
 - Local state is managed within an individual component. For simple applications, this can often be sufficient.
- **Context API:**
 - The Context API is used for more global state management within React applications, avoiding prop drilling for deeply nested components.
- **Other State Management Libraries:**
 - Several libraries such as Redux, MobX, and Recoil provide more comprehensive solutions for state management across larger applications.
 - These tools offer various features suited to different use cases, improving scalability and maintainability of the applications.

Recap: useState hook

- The **useState** hook is a fundamental hook in React that allows you to add React state to functional components.
- The **useState** hook enables you to store state in a function component.
- This hook returns an array containing two elements:
 - Current/Initial State Value
 - State Setter Function / Update Function

Using the useState hook

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  const increment = () => {
    setCount(count + 1);
  };

  return (
    <div>
      <Text>Count: {count}</Text>
      <Button title="Increment" onPress={increment} />
    </div>
  );
}
```

- The useState hook initializes the count state variable to 0.
- The increment function increases the count by 1 and calls setCount to update the state.
- The component re-renders whenever the state changes, displaying the updated count value.

State Management Challenges

- Some limitations of local states are **Prop Drilling** and **State Synchronization**
- Prop drilling refers to the process of passing down props through multiple layers of components, even when some of those components do not directly use the props.
- State synchronization involves ensuring that the data passed to multiple components remains consistent with the updated data source, mostly a state.
 - This is important in cases where update to the state can occur in any of the components

Prop Drilling

- Prop Drilling can lead to challenges like
 - debugging headaches,
 - unexpected behavior from prop mutations,
 - Difficulty in reusing components that are tightly coupled
- Consider a scenario where you have a top-level component that fetches data from an API and needs to pass this data down to multiple nested child components.

Example of Prop Drilling

```
const ParentComponent = () => {  
  // Assume fetching data from an API  
  const data = fetchData();  
  return ( <ChildComponentA data={data} /> );  
};
```

```
const ChildComponentA = ({ data }) => {  
  return ( <ChildComponentB data={data} /> );  
};
```

```
const ChildComponentB = ({ data }) => {  
  return ( <ChildComponentC data={data} /> );  
};
```

```
const ChildComponentC = ({ data }) => {  
  return (<Text>{data.name}</Text>);  
};
```

Example of State Synchronization

```
const ParentComponent = () => {  
  // Assume fetching data from an API  
  const [name, setName] = useState("");  
  return ( <ChildComponentA name={name} updateName={setName} /> );  
};
```

```
const ChildComponentA = ({ name, updateName }) => {  
  return ( <ChildComponentB name={name} updateName={updateName} /> );  
};
```

```
const ChildComponentB = ({ name , updateName }) => {  
  return ( <ChildComponentC data={name} updateName={updateName} /> );  
};
```

```
const ChildComponentC = ({ name , updateName }) => {  
  return (<View>  
    <Text>{name}</Text>  
    <TextInput placeholder="Enter Name" value={name} onChangeText={updateName} />  
  </View>);  
};
```

Advanced State Management

- For larger applications state management in React Native goes beyond simply using the **useState** hook for local state within components.
- Data needs to be shared and synchronized across multiple components.
- The ideal library depends on your project's specific needs such as complexity, scale, developer experience, etc

Advanced State Management: Context API

- Context API is a built-in feature in React.
- It provides a way to share data across components without explicitly passing props down through the component tree (prop drilling).
- It offers a lightweight approach for specific advanced state management scenarios in React Native.
- Common scenarios include managing themes, user settings, or authentication status across an app.

Using Context API

1. Create the context:

- Use `createContext` to create a context object. This object will hold the state data and functions to interact with it.
- Define the initial state value as needed for your application.

```
import { createContext } from 'react';  
  
export const MyContext = createContext("");
```

Using Context API

2. Provider Component:

- Wrap your application's root component with a Provider component.
- Pass the context object and its initial state value as props to the Provider.

```
import React, { useState } from 'react';
import { View } from 'react-native';
import MyContext from './MyContext';
import MyComponent from './MyComponent';

const App = () => {
  const [text, setText] = useState('');

  return (
    <View>
      <MyContext.Provider value={{ text, setText }}>
        <MyComponent />
      </MyContext.Provider>
    </View>
  );
};

export default App;
```

Using Context API

3. Consuming the Context:

- In any component within the provider hierarchy, use the useContext hook with the created context object to access the state and update functions.

```
import { useContext } from 'react';
import { Text, Button, View } from 'react-native';
import MyContext from './MyContext';

const MyComponent = () => {
  const { text, setText } = useContext(MyContext);

  return (
    <View>
      <Text>{text}</Text>
      <Button title="Click me" onPress={() => setText('Hello, world!')} />
    </View>
  );
};

export default MyComponent;
```

Multiple Context Objects

- React allows the creation and use of multiple context objects.
- The parent component is wrapped one after the other with each context provider

Example: Multiple Context Objects

// Create two Contexts

```
const UserContext = createContext();
```

```
const ThemeContext = createContext();
```

// A component that provides both Contexts

```
function App() {
```

```
  return (
```

```
    <UserContext.Provider value="John Doe">
```

```
      <ThemeContext.Provider value="dark">
```

```
        <MyComponent />
```

```
      </ThemeContext.Provider>
```

```
    </UserContext.Provider>
```

```
  );
```

```
}
```

Overriding Context

- You can override the context for a part of the tree (i.e. for other child components) by wrapping that part in a provider with a different value.

```
<ThemeContext.Provider value="dark">  
  <Header />  
  <MyComponent />  
  <ThemeContext.Provider value="light">  
    <Footer />  
  </ThemeContext.Provider>  
</ThemeContext.Provider>
```

Assignment

- Find assignments in Sakai